



Arrays in Javascript

1. JS Objects
2. JS built in objects - array
3. Basic array functionality
4. Advanced array functionality

- Basics
 - non-primitive data type
 - consists of key-value pair where values can be of any data type or even a function.
 - properties can be added or removed dynamically
- Creating objects
 - Can be created using object literal syntax which used pair of curly braces
 - It may include multiple name-value pairs
 - `let person1 = {
 name: "Bakul",
 age: 46
}`

- Accessing object's properties
 - Object's property can be accessed using it's key by 2 different ways
 - Using dot notation: `object.key`
 - Using bracket notation: `object["key"]`
- Nested objects
 - An object can contain value which itself is another object
 - Example: Student can contain marks property whose value is an object
- Objects containing functions
 - an object can also contain a function.
 - can be called using dot notation

- Basics
 - An array is ordered list of element
 - Can have mixed type of values
 - Can grow or shrink dynamically
 - Array consists of array elements accessible using index
- Creating arrays
 - Can be created by 2 different ways:
 - I. using array constructor
 - II. using array literal: Preferred way
- Accessing array elements
 - For accessing array element zero based index should be used
 - Basic syntax: `arrayname[index]`

- Modify array element
 - A new value can be assigned to existing array element which is accessed using it's index
 - `arrayname[index]=new_value`
- Get array size
 - `length` property of an array returns the number of elements.
- Iterating hrough the array
 - 3 different ways
 - I. `for` loop
 - II. `for..in` loop
 - III. `for..of` loop

- Converting array to string
 - `toString()`
 - `join()`
- Adding and removing element from the end
 - `push()`
 - `pop()`
- Adding and removing element from the beginning
 - `unshift()`
 - `shift()`
- Adding and removing element from any position
 - `splice()`
- Slicing array
 - `slice()`

- Concatenating array
 - `concat()`
- Searching element in the array
 - `indexOf()`
 - `lastIndexOf()`
- Reversing the array
 - `reverse()`

- Some functions in array needs another function as argument
- Examples of such functions are: forEach, map, reduce, filter etc
- Callback function
 - Argument to these functions can be passed as function name, anonymous function or arrow function.
 - They are called callback functions because they are not directly called.
- Sorting an array
 - `sort()`
 - Natural sorting: Default ascending based on ascii values
 - Optional argument as in the form of compare function which takes 2 arguments as array elements and returns the difference between them

- Iterating an array
 - `forEach()`
 - accepts one argument as in the form of function which will be called once for each element
- Transforming an array
 - `map()`
 - returns new array where every array element is transformed to the value mentioned in the callback function
 - callback function gets called for every element
- Converting array to single value
 - `reduce()`
 - returns single value which is referred as accumulator value to be used in the next call to callback function
 - callback function gets called for every element

- Filtering an array
 - `filter()`
 - returns new array containing array elements matching the given condition
 - callback function gets called for every element
- Testing array elements for given condition
 - `every()`
 - returns boolean true only when every array element passes the condition given in the callback function otherwise false
 - It gets called for every element